```
# ================================================================
# VARIATIONAL AUTOENCODER (VAE) FOR IMAGE GENERATION
# Dataset: CIFAR-10 (Reliable replacement for CelebA)
# Platform: Google Colab
# ================================================================

import os
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras import layers, Model
from tensorflow.keras.datasets import cifar10

# -------------------- 1. CONFIGURATION --------------------

SEED = 42
tf.keras.utils.set_random_seed(SEED)

IMAGE_SIZE = 32
CHANNELS = 3
LATENT_DIM = 128
BATCH_SIZE = 128
EPOCHS = 20
NUM_GENERATED = 16

print("TensorFlow version:", tf.__version__)
print("Available GPU:", tf.config.list_physical_devices("GPU"))

# -------------------- 2. LOAD CIFAR-10 DATASET ------------------
# Using CIFAR-10 as a reliable alternative to CelebA
print("Loading CIFAR-10 dataset...")

(x_train, _), (x_test, _) = cifar10.load_data()

# Normalize images to [0, 1]
x_train = x_train.astype("float32") / 255.0

# We will use 10,000 images for a quick training demonstration
x_train_subset = x_train[:10000]

train_ds = (
    tf.data.Dataset.from_tensor_slices(x_train_subset)
    .shuffle(2000, seed=SEED)
    .batch(BATCH_SIZE)
    .prefetch(tf.data.AUTOTUNE)
)
```

```
)

print("CIFAR-10 loaded successfully.")
print("Selected training images:", len(x_train_subset))

# -------------------- 3. DISPLAY SAMPLE IMAGES ------------------

sample_batch = next(iter(train_ds))
plt.figure(figsize=(8, 8))

for i in range(16):
    plt.subplot(4, 4, i + 1)
    plt.imshow(sample_batch[i].numpy())
    plt.axis("off")

plt.suptitle("Sample CIFAR-10 Images")
plt.tight_layout()
plt.show()

# -------------------- 4. SAMPLING LAYER --------------------

class Sampling(layers.Layer):
    def call(self, inputs):
        z_mean, z_log_var = inputs
        epsilon = tf.random.normal(
            shape=tf.shape(z_mean)
        )
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon

# -------------------- 5. BUILD ENCODER --------------------

encoder_inputs = layers.Input(
    shape=(IMAGE_SIZE, IMAGE_SIZE, CHANNELS),
    name="encoder_input"
)

x = layers.Conv2D(
    32, 3, strides=2, padding="same", activation="relu"
)(encoder_inputs)

x = layers.Conv2D(
    64, 3, strides=2, padding="same", activation="relu"
)(x)

x = layers.Conv2D(
    128, 3, strides=2, padding="same", activation="relu"
)(x)

x = layers.Flatten()(x)
```

```
x = layers.Dense(256, activation="relu")(x)

z_mean = layers.Dense(LATENT_DIM, name="z_mean")(x)
z_log_var = layers.Dense(LATENT_DIM, name="z_log_var")(x)

z = Sampling()([z_mean, z_log_var])

encoder = Model(
    encoder_inputs,
    [z_mean, z_log_var, z],
    name="encoder"
)

encoder.summary()

# -------------------- 6. BUILD DECODER --------------------

latent_inputs = layers.Input(
    shape=(LATENT_DIM,),
    name="latent_input"
)

x = layers.Dense(4 * 4 * 128, activation="relu")(latent_inputs)
x = layers.Reshape((4, 4, 128))(x)

x = layers.Conv2DTranspose(
    128, 3, strides=2, padding="same", activation="relu"
)(x)

x = layers.Conv2DTranspose(
    64, 3, strides=2, padding="same", activation="relu"
)(x)

x = layers.Conv2DTranspose(
    32, 3, strides=2, padding="same", activation="relu"
)(x)

decoder_outputs = layers.Conv2D(
    CHANNELS, 3, padding="same", activation="sigmoid",
    name="generated_image"
)(x)

decoder = Model(
    latent_inputs,
    decoder_outputs,
    name="decoder"
)

decoder.summary()
```

```
# -------------------- 7. BUILD VAE --------------------

class VAE(Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder

        self.total_loss_tracker = tf.keras.metrics.Mean(
            name="loss"
        )
        self.reconstruction_loss_tracker = tf.keras.metrics.Mean(
            name="reconstruction_loss"
        )
        self.kl_loss_tracker = tf.keras.metrics.Mean(
            name="kl_loss"
        )

    @property
    def metrics(self):
        return [
            self.total_loss_tracker,
            self.reconstruction_loss_tracker,
            self.kl_loss_tracker
        ]

    def train_step(self, data):
        if isinstance(data, tuple):
            data = data[0]

        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(
                data, training=True
            )

            reconstruction = self.decoder(
                z, training=True
            )

            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(
                    tf.square(data - reconstruction),
                    axis=[1, 2, 3]
                )
            )

            kl_loss = -0.5 * tf.reduce_mean(
```

```
                tf.reduce_sum(
                    1 + z_log_var
                    - tf.square(z_mean)
                    - tf.exp(z_log_var),
                    axis=1
                )
            )

            total_loss = reconstruction_loss + kl_loss

        gradients = tape.gradient(
            total_loss, self.trainable_weights
        )

        self.optimizer.apply_gradients(
            zip(gradients, self.trainable_weights)
        )

        self.total_loss_tracker.update_state(total_loss)
        self.reconstruction_loss_tracker.update_state(
            reconstruction_loss
        )
        self.kl_loss_tracker.update_state(kl_loss)

        return {
            "loss": self.total_loss_tracker.result(),
            "reconstruction_loss":
                self.reconstruction_loss_tracker.result(),
            "kl_loss": self.kl_loss_tracker.result()
        }

vae = VAE(encoder, decoder, name="cifar_vae")
vae.compile(optimizer=tf.keras.optimizers.Adam(1e-3))

# -------------------- 8. TRAIN THE VAE --------------------

history = vae.fit(
    train_ds,
    epochs=EPOCHS,
    verbose=1
)

# -------------------- 9. PLOT TRAINING LOSSES --------------------

plt.figure(figsize=(10, 5))

plt.plot(
    history.history["loss"],
    label="Total Loss",
```

```
    marker="o"
)

plt.plot(
    history.history["reconstruction_loss"],
    label="Reconstruction Loss",
    marker="o"
)

plt.plot(
    history.history["kl_loss"],
    label="KL Divergence",
    marker="o"
)

plt.title("VAE Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# -------------------- 10. COMPARE ORIGINAL AND RECONSTRUCTED ----

original_images = sample_batch[:NUM_GENERATED]

_, _, latent_vectors = encoder.predict(
    original_images, verbose=0
)

reconstructed_images = decoder.predict(
    latent_vectors, verbose=0
)

plt.figure(figsize=(12, 6))

for i in range(NUM_GENERATED):
    plt.subplot(2, NUM_GENERATED, i + 1)
    plt.imshow(original_images[i].numpy())
    plt.axis("off")
    if i == 0:
        plt.ylabel("Original", fontsize=12)

    plt.subplot(2, NUM_GENERATED, NUM_GENERATED + i + 1)
    plt.imshow(np.clip(reconstructed_images[i], 0, 1))
    plt.axis("off")
    if i == 0:
        plt.ylabel("Reconstructed", fontsize=12)
```

```
plt.suptitle("Original vs VAE-Reconstructed Images")
plt.tight_layout()
plt.show()

# -------------------- 11. GENERATE NEW IMAGES --------------------

random_latent_vectors = tf.random.normal(
    shape=(NUM_GENERATED, LATENT_DIM),
    seed=SEED
)

generated_images = decoder.predict(
    random_latent_vectors,
    verbose=0
)

plt.figure(figsize=(8, 8))

for i in range(NUM_GENERATED):
    plt.subplot(4, 4, i + 1)
    plt.imshow(np.clip(generated_images[i], 0, 1))
    plt.axis("off")

plt.suptitle("New Images Generated from the Learned Latent Distribu
plt.tight_layout()
plt.show()

# -------------------- 12. SAVE THE MODELS --------------------

encoder.save("/content/vae_encoder.keras")
decoder.save("/content/vae_decoder.keras")

vae.save_weights("/content/vae_weights.weights.h5")

print("VAE training completed.")
print("Encoder saved: /content/vae_encoder.keras")
print("Decoder saved: /content/vae_decoder.keras")
print("Weights saved: /content/vae_weights.weights.h5")

# -------------------- 13. DOWNLOAD GENERATED IMAGES --------------

from PIL import Image as PILImage

os.makedirs("/content/generated_images", exist_ok=True)

for i, image in enumerate(generated_images):
    image_uint8 = (
```

```
        np.clip(image, 0, 1) * 255
    ).astype(np.uint8)

    PILImage.fromarray(image_uint8).save(
        f"/content/generated_images/generated_img_{i+1}.png"
    )

print("Generated images saved in /content/generated_images/")
print("Project finished successfully!")
```

```
TensorFlow version: 2.21.0
Available GPU: []
Loading CIFAR-10 dataset...
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-py
161005568/170498071 ━━━━━━━━━━━━━━━━━━━ 3:00 19us/step
```